

UNIFAL-MG
CIÊNCIA DA COMPUTAÇÃO

Gustavo Borin Nascimento

**Validação de Atributos Privados em Contratos Inteligentes via
Zero-Knowledge Proofs: Implementação e Benchmarking de Range
Proofs utilizando a Linguagem Noir**

Orientador: Iago Augusto de Carvalho

Alfenas, 26 de maio de 2026

1. IDENTIFICAÇÃO DO PROJETO

Campo	Informação
Título	Validação de Atributos Privados em Contratos Inteligentes via Zero-Knowledge Proofs: Implementação e Benchmarking de Range Proofs utilizando a Linguagem Noir
Título em inglês	Validation of Private Attributes in Smart Contracts via Zero-Knowledge Proofs: Implementation and Benchmarking of Range Proofs using Noir
Aluno	Gustavo Borin Nascimento
Orientador	Iago Augusto de Carvalho
Curso	CIÊNCIA DA COMPUTAÇÃO
Data de Submissão	26/05/2026

2. RESUMO

A crescente adoção de redes públicas de blockchain, em especial o Ethereum, impõe uma tensão fundamental entre transparência computacional e privacidade de dados. O modelo de consenso baseado em transições de estado verificáveis por todos os participantes exige a exposição dos dados transacionados, tornando inviável a validação de condições sensíveis — como solvência financeira ou conformidade regulatória — sem revelar informações confidenciais.

O presente trabalho propõe a implementação e o benchmarking empírico de um circuito criptográfico do tipo Range Proof utilizando a linguagem de domínio específico Noir, com backend UltraHonk fornecido pela biblioteca Barretenberg. O circuito permitirá provar matematicamente, de forma verificável on-chain na Ethereum Virtual Machine (EVM), que um valor privado v pertence a um intervalo válido $a \leq v \leq b$, sem revelar o valor exato de v .

A metodologia emprega um delineamento experimental controlado, comparando quatro variantes do circuito (tipos Field, $u8$, $u32$ e $u64$) em dois ambientes de execução: nativo (CLI Barretenberg) e cliente (WASM32 via V8/Chrome). O desempenho é avaliado pelo tempo de geração da prova (proving time), tamanho do artefato, contagem de restrições (ACIR opcodes e backend gates) e custo de Gas na EVM pós-Pectra, considerando os EIPs 170, 4844 e 7623. O controle estatístico emprega o teste de Brunner-Munzel com correção de Holm-Bonferroni para múltiplas comparações.

A contribuição esperada é um estudo empírico original que quantifica o custo da privacidade em contratos inteligentes EVM no ecossistema Noir/Barretenberg, preenchendo uma lacuna identificada na literatura de benchmarking de sistemas ZK, onde trabalhos com esse perfil — combinando DSL moderna, pipeline ACIR + UltraHonk, e instrumentação de Gas pós-Pectra — são inexistentes.

Palavras-chave: Zero-Knowledge Proofs; Range Proof; Noir; UltraHonk; Benchmarking; Ethereum; WebAssembly.

2.1 ABSTRACT

The growing adoption of public blockchain networks, especially Ethereum, introduces a fundamental tension between computational transparency and data privacy. The consensus model based on state transitions verifiable by all participants requires the exposure of transacted data, making it unfeasible to validate sensitive conditions — such as financial solvency or regulatory compliance — without disclosing confidential information.

This work proposes the implementation and empirical benchmarking of a Range Proof cryptographic circuit using the domain-specific language Noir, with the UltraHonk backend provided by the Barretenberg library. The circuit will allow mathematically proving, verifiably on-chain in the Ethereum Virtual Machine (EVM),

that a private value v belongs to a valid interval $a \leq v \leq b$, without revealing the exact value of v .

The methodology employs a controlled experimental design, comparing four circuit variants (types Field, $\mathbb{F}_8$, $\mathbb{F}_{32}$, and $\mathbb{F}_{64}$) in two execution environments: native (Barretenberg CLI) and client-side (WASM32 via V8/Chrome). Performance is assessed via proving time, artifact size, constraint count (ACIR opcodes and backend gates), and Gas cost on the post-Pectra EVM, accounting for EIPs 170, 4844, and 7623. Statistical control employs the Brunner-Munzel test with Holm-Bonferroni correction for multiple comparisons.

The expected contribution is an original empirical study quantifying the cost of privacy in EVM smart contracts within the Noir/Barretenberg ecosystem, filling a gap identified in the ZK benchmarking literature where works combining a modern DSL, ACIR + UltraHonk pipeline, and post-Pectra Gas instrumentation are nonexistent.

Keywords: Zero-Knowledge Proofs; Range Proof; Noir; UltraHonk; Benchmarking; Ethereum; WebAssembly.

3. INTRODUÇÃO E CONTEXTUALIZAÇÃO DO PROBLEMA

O Ethereum, desde sua concepção, opera como uma máquina de estados distribuída e transparente. Cada transação submetida à rede é executada por todos os nós validadores, que verificam independentemente a transição de estado e chegam ao mesmo resultado determinístico. Essa propriedade de transparência computacional é o alicerce da segurança e da resistência à censura que tornaram o Ethereum a principal plataforma de smart contracts em operação. Contudo, essa mesma transparência impõe uma restrição estrutural severa: toda condição avaliada por um contrato inteligente exige a exposição dos dados que a fundamentam. O calldata enviado nas transações e o estado de armazenamento dos contratos são registrados permanentemente na cadeia pública, acessíveis a qualquer observador.

Essa característica cria uma tensão fundamental em aplicações com requisitos de confidencialidade. Para que um contrato inteligente valide, por exemplo, que o saldo de um usuário supera um limite mínimo de solvência, ou que um indicador de risco se enquadra em uma faixa de conformidade regulatória, o usuário é atualmente compelido a revelar seu saldo exato ou seu histórico financeiro completo à rede pública. Em contextos financeiros e regulatórios, essa exposição viola normas de sigilo bancário, legislações de proteção de dados e, em muitos casos, representa um risco competitivo inaceitável para as partes envolvidas. A verificação computacional distribuída, portanto, é obtida ao custo da confidencialidade.

As Zero-Knowledge Proofs (ZKPs) surgem como a solução criptográfica para essa tensão. Formalmente fundamentadas na teoria de complexidade de prova por Goldwasser, Micali e Rackoff, ZKPs permitem que um provador convença um verificador da veracidade de um enunciado sem revelar qualquer informação além do fato de que o enunciado é verdadeiro. Dentre as primitivas ZK aplicáveis a contratos inteligentes, as Range Proofs são especialmente relevantes: permitem provar que um valor oculto v pertence a um intervalo $[a, b]$, satisfazendo a relação $a \leq v \leq b$, sem expor v . No ecossistema Ethereum, SNARKs modernos — como os gerados pelo pipeline Noir/UltraHonk — permitem que essa prova seja verificada on-chain por um contrato inteligente com custo de Gas fixo e tamanho de prova sucinto, independente da complexidade do witness subjacente.

O ecossistema Noir, desenvolvido pela Aztec Labs, representa uma das fronteiras mais ativas nesse campo. Noir é uma linguagem de domínio específico que abstrai a complexidade criptográfica, permitindo expressar lógica de prova em sintaxe de alto nível enquanto o compilador gera Abstract Circuit Intermediate Representation (ACIR), consumida pelo backend Barretenberg para produzir provas UltraHonk. UltraHonk é um protocolo PLONKish avançado que substitui o polinômio quociente global por um protocolo de Sumcheck sobre o hipercubo booleano, otimizando uso de memória para circuitos grandes. Apesar da maturidade crescente desse ecossistema, há uma lacuna significativa na literatura: inexistem estudos de benchmarking sistemático que combinem (i) a análise do impacto de diferentes tipos numéricos ($\mathbb{U8}$, $\mathbb{U32}$, $\mathbb{U64}$ versus Field) sobre o custo do provador, (ii) a

comparação empírica entre execução nativa e WASM32 com controle de threads e térmica, e (iii) a instrumentação de Gas na EVM sob os EIPs 170, 4844 e 7623, introduzidos pelo fork Pectra. Essa lacuna é documentada na Seção 6 (Revisão Bibliográfica) deste documento, especialmente na subseção 2.5.

O presente trabalho propõe preencher essa lacuna por meio de uma pesquisa aplicada de engenharia de software: projetar, implementar e mensurar empiricamente o desempenho de um circuito de Range Proof em Noir com quatro variantes de tipo numérico, submetendo-o a um protocolo experimental rigoroso descrito na Seção 7 (Metodologia). Os objetivos específicos, detalhados na Seção 4.2, incluem a implementação das variantes do circuito, a coleta de dados de proving time em ambos os ambientes de execução, a instrumentação de Gas em fork local da Sepolia via anvil, e a análise estatística com teste de Brunner-Munzel e correção de Holm-Bonferroni. A contribuição esperada é um conjunto de dados empíricos originais e uma análise quantitativa do custo da privacidade no ecossistema Noir/Barretenberg/EVM, servindo de referência para pesquisadores e engenheiros que avaliem a adoção de ZKPs em aplicações client-side e on-chain.

4. OBJETIVOS

4.1 Objetivo Geral

Projetar, implementar e avaliar empiricamente o desempenho de um circuito criptográfico do tipo Range Proof construído na linguagem Noir com backend UltraHonk (Barretenberg), quantificando o custo computacional e a degradação de desempenho em função da largura de bits (\$u8\$, \$u32\$, \$u64\$ versus Field) e do ambiente de execução (nativo versus WASM32), bem como o custo de verificação on-chain na EVM pós-Pectra, de modo a fornecer evidências empíricas originais sobre o trade-off entre privacidade algorítmica e custo operacional no ecossistema Noir/Barretenberg/Ethereum.

4.2 Objetivos Específicos

1. Projetar e implementar o circuito Range Proof em Noir com quatro variantes de tipo numérico (Field, \$u8\$, \$u32\$, \$u64\$), garantindo equivalência funcional entre as variantes para permitir comparação isolada do custo da restrição de intervalo.
2. Realizar o benchmarking do proving time em ambiente nativo (CLI Barretenberg) e em ambiente WASM32 (navegador Chrome/V8) com controle simétrico de threads, afinidade de CPU via taskset e fixação do governor de frequência para expurgar variâncias térmicas e de tiering JIT.
3. Instrumentar o custo de Gas — execution gas, calldata gas (EIP-7623) e blob gas (EIP-4844, como projeção teórica) — no fork local da rede de testes Sepolia via anvil, operando sobre o bloco mais recente para garantir equivalência com o ambiente pós-Pectra.
4. Avaliar a complexidade do circuito para cada variante por meio da contagem determinística de ACIR Opcodes (via nargo info) e Backend Gates (via bb gates), extraindo a topologia real fornecida ao Barretenberg.
5. Verificar a correção lógica das quatro variantes do circuito por meio de testes negativos (Under-constrained Testing), submetendo inputs fora do intervalo declarado para assegurar ausência de vulnerabilidades de subconstrição antes da coleta de desempenho.
6. Quantificar o gap de desempenho entre execução nativa e WASM32, e entre as variantes de tipo numérico, aplicando o teste estatístico não paramétrico de Brunner-Munzel com correção de Holm-Bonferroni para múltiplas comparações, adotando a probabilidade estocástica $P(X < Y)$ como métrica formal de tamanho do efeito.
7. Sintetizar e disponibilizar os dados empíricos coletados, os scripts de experimentação e os artefatos de circuito como contribuição reprodutível para a comunidade de pesquisa em sistemas ZK aplicados ao ecossistema Ethereum.

5. JUSTIFICATIVA

A relevância prática deste trabalho é imediata e multifacetada. A crescente adoção de finanças descentralizadas (DeFi), sistemas de conformidade regulatória on-chain e protocolos de identidade digital em redes públicas como o Ethereum cria uma demanda urgente por mecanismos que permitam validar atributos sensíveis sem comprometer a confidencialidade dos dados subjacentes. Range Proofs são a primitiva criptográfica central nessa categoria de aplicações: permitem provar solvência sem revelar saldo, demonstrar elegibilidade regulatória sem expor histórico financeiro, e verificar conformidade de parâmetros de risco sem divulgar estratégias de trading. A quantificação empírica do custo operacional — em proving time e Gas — dessas provas no ecossistema Ethereum é, portanto, um insumo direto para decisões de engenharia em produtos financeiros e de conformidade baseados em blockchain.

Do ponto de vista técnico, o ecossistema Noir/UltraHonk representa uma fronteira recente e pouco estudada empiricamente. A linguagem Noir, mantida pela Aztec Labs, tornou-se uma das DSLs ZK de maior adoção no ecossistema Ethereum, especialmente após a abertura do pipeline Barretenberg como biblioteca independente. O protocolo UltraHonk, com sua arquitetura baseada em Sumcheck e lookup tables, difere substancialmente dos sistemas Groth16 e PLONK originais que dominam a literatura de benchmarking. A ausência de dados empíricos específicos para esse pipeline — especialmente considerando a variável do ambiente de execução WASM32, predominante em aplicações client-side — representa uma lacuna técnica significativa que impede que engenheiros façam projeções fundamentadas de desempenho para sistemas ZK baseados em Noir.

O gap de pesquisa é explicitamente documentado na subseção 2.5 da Revisão Bibliográfica (Seção 6 deste documento). Os trabalhos de Benarroch et al. (2020), Ernstberger et al. e Sheybani et al. (2025) fornecem frameworks e surveys valiosos de benchmarking para sistemas ZK, mas focam em Groth16, PLONK, Halo2 ou frameworks genéricos, frequentemente em ambiente exclusivamente nativo. O trabalho de Derei (2023) aborda aceleração do provador PLONK via GPU, mas não examina o pipeline ACIR + UltraHonk nem o ambiente WASM32. Nenhum trabalho localizado combina os três eixos de análise deste TCC: (i) impacto de tipos numéricos (\$u8\$, \$u32\$, \$u64\$ versus Field) sobre o provador Noir/UltraHonk, (ii) comparação nativo versus WASM32 com controle experimental rigoroso, e (iii) instrumentação de Gas na EVM pós-Pectra com os EIPs 170, 4844 e 7623.

A originalidade metodológica deste trabalho reside precisamente na combinação de rigor experimental com especificidade técnica. O protocolo descrito na Seção 7 (Metodologia) incorpora controles que são raros ou inexistentes na literatura: fixação simétrica de threads entre ambientes nativo e WASM32 usando `navigator.hardwareConcurrency` como âncora comum; controle térmico via `cpupower` e supressão de Turbo Boost para eliminar variância de throttling; protocolo de burn-in de 15 iterações para forçar o tier-up do compilador V8 (Liftoff para TurboFan)

antes da coleta; timeout dinâmico baseado na mediana do burn-in para isolar falhas de ambiente de lentidão criptográfica genuína; e análise de dominância estocástica via Brunner-Munzel, adotando $P(X < Y)$ como métrica de tamanho do efeito — abordagem especialmente adequada à natureza multimodal da latência em ambientes WASM32. Essa combinação de controles assegura que os resultados obtidos reflitam o comportamento real do sistema em condições de uso comparáveis, e não artefatos de ambiente.

6. REVISÃO BIBLIOGRÁFICA

2.1 Fundamentos de Zero-Knowledge Proofs

O conceito de zero-knowledge proofs (ZKPs) surge na teoria de complexidade de prova, a partir da tentativa de quantificar quanta "informação" além da veracidade de um enunciado é transmitida em um protocolo de prova. Goldwasser, Micali e Rackoff introduziram a noção de **complexidade de conhecimento** e definiram formalmente provas de conhecimento zero, mostrando que é possível convencer um verificador da veracidade de certas propriedades sem revelar o witness subjacente. Um exemplo clássico discutido pelos autores é o problema da residuidade quadrática, em que o provador demonstra que um número é resíduo quadrático módulo um composto sem revelar a raiz quadrada em si.[1][2][3]

Formalmente, uma zero-knowledge proof é um protocolo interativo entre um provador P e um verificador V para uma linguagem $L \subseteq \{0,1\}^*$ em NP, associada a uma relação $R(x,w)$ tal que $x \in L$ se, e somente se, existe um witness w com $R(x,w) = 1$. O protocolo deve satisfazer três propriedades fundamentais: **completude**, **solidez** (soundness) e **conhecimento zero**. [4][1]

- **Completude**: para todo $x \in L$ e witness válido w , um provador honesto consegue convencer um verificador honesto com probabilidade próxima de 1. [1][4]
- **Solidez**: para $x \notin L$, nenhum provador malicioso sem acesso a um witness consegue convencer um verificador honesto, exceto com probabilidade negligenciável. [4][1]
- **Conhecimento zero**: existe um simulador eficiente que, sem acesso ao witness, gera transcrições indistinguíveis das obtidas na interação real, garantindo que o verificador não aprende nada além do fato de que $x \in L$. [2][1]

No contexto deste TCC, o circuito de Range Proof em Noir implementa precisamente esse paradigma: existe uma relação R que codifica " $a \leq v \leq b$ " sobre um valor privado v , e o provador apresenta uma prova UltraHonk que convence o contrato verificador na EVM de que $R(x,w)=1$, sem revelar v . As propriedades de completude, solidez e conhecimento zero são herdadas do sistema de prova UltraHonk, implementado no backend Barretenberg, que serve como motor criptográfico para Noir. [5][6][7][8][9]

Provas interativas, NIZKs e transformação de Fiat–Shamir

As ZKPs foram inicialmente formuladas como **protocolos interativos**, em que verificador e provador trocam múltiplas mensagens, com o verificador escolhendo desafios aleatórios a cada rodada. Em ambientes distribuídos como blockchains, entretanto, é desejável eliminar essa interação, substituindo-a por uma prova única que o verificador possa checar off-line: as chamadas **Non-Interactive Zero-Knowledge Proofs (NIZKs)**. [10][11][1][4]

A **transformação de Fiat–Shamir** é um mecanismo heurístico que converte protocolos públicos de três mensagens em provas não interativas, substituindo os desafios aleatórios do verificador por chamadas a uma função hash modelada como random oracle. Dado um transcript parcial t , o desafio c é definido como $c = H(t)$, onde H é uma função hash criptográfica, eliminando a necessidade de um verificador ativo durante a prova. Essa técnica é usada tanto para derivar NIZKs quanto para construir esquemas de assinatura a partir de protocolos de identificação.[11][12][10]

Sistemas NIZK modernos introduzem ainda o conceito de **Common Reference String (CRS)**, uma string pública compartilhada entre provador e verificador, gerada em um procedimento de setup anterior. Esquemas como Groth16 e PLONK dependem de um CRS estruturado, obtido por meio de uma cerimônia de trusted setup multiparte, que garante que parâmetros secretos usados no compromisso polinomial não sejam conhecidos por nenhuma parte isolada.[13][14][15][4]

SNARKs e STARKs

Os Succinct Non-interactive ARguments of Knowledge (SNARKs) são NIZKs com provas de tamanho pequeno e tempo de verificação sublinear no tamanho do witness, propriedades cruciais para aplicações em blockchains. O esquema Groth16, por exemplo, oferece provas de tamanho constante (três elementos em um grupo de emparelhamento) e verificação muito rápida, à custa de um trusted setup específico para cada circuito. Isso o tornou um padrão de facto em muitas aplicações de privacidade e rollups baseados em emparelhamentos.[14][16][13][4]

PLONK introduz uma família de SNARKs universais e atualizáveis, baseados em compromissos polinomiais e em um **argumento de permutação** sobre avaliações em uma base de Lagrange. Em vez de um setup por circuito, PLONK utiliza um CRS universal reutilizável para múltiplos programas, simplificando o desenvolvimento de ecossistemas de contratos ZK. Extensões "PLONKish" agregam lookup tables, custom gates e otimizações para recursão, permitindo expressar operações inteiras, comparações e range checks com menos gates do que em circuitos puramente aritméticos.[15][17][18][19][20]

Os Scalable Transparent Arguments of Knowledge (STARKs) são argumentos transparentes (sem trusted setup) baseados sobretudo em funções hash, códigos de correção de erros e protocolos FRI. Nesse modelo, a segurança se baseia em suposições de resistência a colisões de hash e em argumentos de correção de códigos, e não em emparelhamentos bilineares. STARKs oferecem boa escalabilidade para grandes volumes de computação, mas produzem provas maiores do que SNARKs emparelhados, o que impacta custos de transmissão e armazenamento em blockchains.[21][22][23][4]

UltraHonk, lookup tables e Sumcheck

No ecossistema Aztec/Noir, a biblioteca Barretenberg implementa uma família PLONKish otimizada, incluindo UltraPlonk e **UltraHonk**, projetados para prover

provas sucintas compatíveis com EVM e circuitos de privacidade de grande escala. A documentação indica que o compilador Noir gera provas UltraHonk ao utilizar Barretenberg como backend, e que contratos verificadores podem ser gerados automaticamente com `bb write_solidity_verifier`.^{[6][8][9][20][5]}

UltraHonk não é apenas "PLONK com lookups". Parte central da família Honk é substituir o polinômio quociente global de PLONK clássico por um protocolo de **Sumcheck** sobre o hipercubo booleano, o que reduz a necessidade de materializar polinômios de alto grau associados a circuitos grandes na memória. Em vez de construir um único polinômio quociente cujo grau cresce com o tamanho do circuito e avaliá-lo em pontos aleatórios, o provador Honk usa Sumcheck para provar que a soma de avaliações de polinômios multilineares sobre $\{0,1\}^n$ satisfaz uma relação esperada, decompondo a prova em rodadas que lidam com uma dimensão de cada vez.^{[9][24][25][26]}

Implementações práticas de Sumcheck mostram que o desempenho tende a ser limitado por largura de banda de memória e padrões de acesso, mais do que por operações de campo isoladas, o que torna essa família particularmente adequada para circuitos grandes em ambientes de memória restrita. Barretenberg explora esse desenho para tornar UltraHonk mais escalável em uso de memória do que um PLONK clássico com polinômio quociente, ainda que com provas ligeiramente maiores. No contexto deste TCC, essa característica é crítica: ao medir proving time em ambientes nativo e WASM32, o experimento captura como um backend baseado em Sumcheck se comporta em arquiteturas com diferentes padrões de cache, latência de memória e limites de heap.^{[25][26][27][28][29][5][9]}

✓ Fontes desta subseção verificadas.

2.2 Contratos Inteligentes e a Ethereum Virtual Machine (EVM)

Nick Szabo introduziu o termo **smart contracts** na década de 1990 para descrever acordos autoexecutáveis cujos termos são implementados diretamente em hardware ou software, em vez de depender exclusivamente de instituições legais. Em textos como *Smart Contracts: Building Blocks for Digital Markets*, Szabo argumenta que cláusulas contratuais, penalidades e condições de execução podem ser embutidas em sistemas computacionais, permitindo automação de transações e redução de custos de agência.^{[30][31][32]}

O Ethereum White Paper, de Vitalik Buterin, propõe uma plataforma de contratos inteligentes sobre uma blockchain Turing-completa, definindo um modelo de máquina de estados global em que cada transação é uma transição determinística aplicada ao estado corrente. O estado Ethereum é composto por contas externas e de contrato, cada uma com saldo, nonce, bytecode e armazenamento; transações modificam esse estado ao acionar o bytecode de contratos na Ethereum Virtual Machine (EVM). Isso garante que todos os nós honestos, ao executar as mesmas transações na mesma ordem, cheguem ao mesmo estado final, estabelecendo consenso. No TCC, o verificador UltraHonk gerado por `bb write_solidity_verifier` é

exatamente um contrato inteligente EVM, responsável por validar provas de Range Proof enviadas como calldata.[28][33][34][35][5][6]

A EVM e o modelo de Gas

A **Ethereum Virtual Machine (EVM)** é uma máquina de pilha de 256 bits com conjunto de instruções próprio, projetada para ser determinística e isolada. Cada instrução consome uma quantidade fixa de **Gas**, uma unidade abstrata de custo computacional e de armazenamento, usada para limitar o consumo de recursos em uma transação e prevenir ataques de negação de serviço. Os usuários definem um preço de Gas em Ether por unidade, e o custo total da transação é dado pelo produto entre Gas consumido e preço de Gas, o que desacopla a contagem de operações da volatilidade do Ether.[33][35]

Para verificadores ZK on-chain, Gas é a métrica principal de custo, pois operações como multiplicações em curvas elípticas, leituras de calldata e escrita em armazenamento possuem custos significativamente distintos. Nesse TCC, o contrato verificador UltraHonk implantado em Sepolia tem seu **execution gas** e o **calldata gas** medidos sistematicamente, sob a tabela de custos da EVM pós-Pectra e levando em conta EIPs como 170, 4844 e 7623.[35][36][37][28]

EIPs relevantes: EIP-170, EIP-4844 e EIP-7623

O **EIP-170** introduz um limite máximo de **24.576 bytes** para o tamanho do bytecode de contratos (`MAX_CODE_SIZE = 0x6000`). Caso o bytecode retornado na criação de um contrato exceda esse limite, a criação falha. A justificativa é mitigar ataques em que contratos extremamente grandes impõem custos desproporcionais de armazenamento e validação aos nós da rede. Verificadores SNARK gerados automaticamente, especialmente para circuitos grandes com muitas constraints e lookup tables, frequentemente produzem bytecode próximo a esse limite, exigindo estratégias como otimização agressiva de compilação ou split-verifier. Este TCC lida explicitamente com essa restrição, ajustando parâmetros de compilação Solidity e, se necessário, adotando topologias particionadas para o verificador UltraHonk em Sepolia.[36][38][5][28][35]

O **EIP-4844** (Proto-Danksharding) introduz transações com **blobs de dados**, anexando grandes blocos de dados a um tipo específico de transação e criando um mercado de taxas separado denominado blob gas. O conteúdo dos blobs é mantido pela camada de consenso por um período limitado, fornecendo disponibilidade de dados para rollups, mas **não é acessível diretamente pela EVM** — contratos só podem observar compromissos criptográficos (por exemplo, via BLOBHASH), não o conteúdo bruto. Arquiteturas de **rollups ZK** lidam com isso publicando apenas compromissos polinomiais (por exemplo, KZG) e provas curtas na L1, enquanto os dados completos são armazenados em blobs usados exclusivamente como camada de disponibilidade. A verificação L1 resume-se a checar provas de avaliação pontual sobre esses compromissos, muitas vezes via precompiles específicos, sem que a EVM leia o blob em si.[39][40][41][42][43]

O verificador UltraHonk deste TCC, porém, é um contrato **autônomo** implantado diretamente na Sepolia, que recebe toda a prova como calldata e executa a verificação completa dentro da EVM, sem o auxílio de uma infraestrutura de rollup ou de precompiles avançadas. Nessa arquitetura, o EIP-4844 funciona apenas como "projeção teórica" para cenários futuros em camada 2, enquanto o custo real de transmissão de provas é governado por calldata, ficando diretamente sujeito ao EIP-7623.[37][44][5][28][35]

O **EIP-7623** propõe **aumentar o custo de calldata** para reduzir o tamanho efetivo de blocos dominados por dados e melhorar a resiliência da rede. A proposta introduz um modelo de cobrança em que bytes de calldata têm custo base maior e um penalty adicional para transações que usam volume extremo de dados, elevando o custo efetivo por byte em até cerca de 150% em casos limite. Como provas ZK são frequentemente enviadas como calldata para contratos verificadores, o EIP-7623 impacta diretamente o custo de prover privacidade via ZKPs na L1. Por isso, o TCC avalia explicitamente o **tamanho da prova em bytes** e o **Gas associado ao calldata** como métricas centrais do custo de privacidade na EVM pós-Pectra.[44][45][46][47][28][37][39]

✓ Fontes desta subseção verificadas.

2.3 Range Proofs: Fundamentos e Aplicações

Uma **Range Proof** é uma prova de conhecimento zero que demonstra que um valor secreto v pertence a um intervalo $[a, b]$, sem revelar v . A formulação típica envolve um compromisso criptográfico a v (por exemplo, um compromisso de Pedersen) e uma prova de que $a \leq v \leq b$, garantindo ao verificador que o valor é não negativo e limitado, sem permitir reconstruir v . Em termos de relações NP, o enunciado público inclui o intervalo $[a, b]$ e o compromisso C , enquanto o witness inclui v e a aleatoriedade do compromisso; a prova convence o verificador de que existe um witness consistente com a relação " $a \leq v \leq b$ e C é um compromisso de v ".[48][49][50][51]

Essa tarefa é criptograficamente não trivial porque o compromisso precisa satisfazer propriedades de ocultação e vinculação (no caso de Pedersen, esconder o valor sob a dificuldade do logaritmo discreto), enquanto a prova de faixa deve impedir que valores negativos ou valores maiores que b sejam aceitos. Em sistemas de transações confidenciais, Range Proofs evitam a criação de moeda do nada e garantem que nenhum valor comprometido representa quantias inválidas, sem expor saldos exatos.[50][52][53][54][48]

Compromissos de Pedersen e Bulletproofs

Compromissos de Pedersen são esquemas aditivos do tipo binding sob hipótese de logaritmo discreto e perfectly hiding, muito usados na construção de Range Proofs. Dados geradores independentes G e H em um grupo de ordem q , um compromisso a um valor v com aleatoriedade r é dado por $C = vG + rH$; sem

saber o logaritmo discreto de H na base G , não é possível abrir C para dois pares (v, r) distintos.[51][52][50]

Construções iniciais de Range Proofs com Pedersen decompõem v em bits e usam compromissos a cada bit, junto de provas de que cada compromisso representa um valor em $\{0,1\}$ e que a combinação desses bits, ponderados por potências de dois, resulta em v . Esse tipo de construção, embora simples, gera provas de tamanho linear no número de bits do intervalo, o que é caro para faixas largas.[49][48]

Bulletproofs, proposta de Bünz et al., introduzem um protocolo de Range Proof não interativo com tamanho **logarítmico** em relação ao número de bits do intervalo, sem necessidade de trusted setup. A técnica se baseia em inner-product arguments recursivos: em vez de provar bit a bit, o provador constrói uma representação vetorial de v e usa uma sequência de desafios para reduzir o problema a um caso base de dimensão pequena. Bulletproofs foram rapidamente adotadas em sistemas de transações confidenciais, pois reduziram dramaticamente o tamanho das provas em comparação com esquemas lineares.[53][55][48][49]

Range Proofs baseadas em SNARKs, como Groth16 ou PLONK, seguem outra abordagem: codificam o predicado " $v \in [a,b]$ " como um circuito aritmético ou sistema de equações polinomiais, e depois usam um SNARK de propósito geral para comprovar a satisfatibilidade desse circuito. A sucintez da prova independe do número de bits do intervalo, permanecendo constante, enquanto o custo de prova é proporcional ao número de constraints do circuito. O verificador UltraHonk utilizado no TCC segue esse paradigma: o Range Proof é um circuito Noir compilado para ACIR, e UltraHonk gera uma prova sucinta independente do tamanho do intervalo, embora o custo de prova varie com a complexidade do circuito.[8][5][13][14][15][28]

Aplicações práticas

Range Proofs são essenciais em **transações confidenciais**, em particular em esquemas de Confidential Transactions e em criptomoedas de privacidade como Monero. O mecanismo Ring Confidential Transactions (RingCT) combina assinaturas em anel, compromissos de Pedersen e Range Proofs para ocultar remetente, destinatário e quantias, assegurando simultaneamente que a soma das entradas iguala a soma das saídas e que nenhum valor é negativo. Nessa arquitetura, Range Proofs típicas garantem que cada compromisso representa um valor em $[0, 2^{64})$, evitando criação de saldos negativos que inflariam o supply.[54][56][48][51][53]

Em **provas de solvência** de exchanges, Range Proofs permitem provar que o patrimônio líquido da instituição é positivo e que reservas superam passivos, sem expor saldos individuais de clientes. Em **conformidade regulatória** (KYC/AML), Range Proofs podem demonstrar que um usuário possui saldo acima de um mínimo, que sua idade é superior a um limite ou que um índice de risco está dentro de uma faixa aceitável, sem divulgar valores exatos. Em **leilões e sistemas de votação**,

Range Proofs podem garantir que lances ou votos pertencem a faixas válidas, protegendo o sigilo de preferências.[57][4]

No TCC, a Range Proof em Noir é usada para provar on-chain que um atributo privado v (por exemplo, um saldo ou um indicador de conformidade) pertence ao intervalo $[a,b]$ exigido pelo contrato inteligente, permitindo validar regras de solvência ou elegibilidade sem expor v . [7][8]

2.3.1 Argumentos de lookup e custo assintótico

Lookup arguments são técnicas que permitem provar, de forma sucinta, que cada elemento de um vetor de consulta provém de uma tabela pré-computada, sem revelar informações adicionais sobre o conteúdo da tabela nem sobre a posição exata de cada elemento. Em termos abstratos, dados dois conjuntos de valores representados como colunas A (consultas) e S (tabela) em uma tabela com 2^k linhas, um argumento de lookup estabelece que $A \subseteq S$, isto é, que cada linha de A coincide com alguma linha de S . [58][59][60][61]

Esquemas PLONKish como Halo2 e variantes UltraPlonk/UltraHonk utilizam argumentos de lookup para implementar operações não aritméticas — checagens de faixa, comparações, funções especiais — de modo mais eficiente do que via polinômios de alto grau. A ideia é deslocar a complexidade de expressar essas operações para a construção de uma tabela de valores válidos e de um argumento que comprove que as saídas produzidas pelo circuito pertencem a essa tabela, em vez de codificar a operação inteira como um único polinômio de alto grau. [59][18][62][58]

O custo dessas otimizações não é gratuito: o provador deve comprometer colunas extras representando a tabela e construir polinômios auxiliares que garantam que cada entrada de consulta aparece em alguma linha da tabela, o que introduz complexidade proporcional ao número de linhas da tabela. Para uma tabela densa de u , o espaço de valores possíveis é 2^u ; para $u=32$ ou $u=64$, tabelas ingênuas teriam 2^{32} ou 2^{64} entradas, o que é impraticável e obriga a decomposições em múltiplas tabelas menores (por exemplo, por limbs de 8 ou 16 bits) ou em bases intermediárias. Assim, o ganho em número de gates aritméticos vem acompanhado de um aumento assintótico no número de linhas de lookup e, consequentemente, em operações de compromisso e verificações associadas ao argumento de lookup. [61][62][63][59]

Backends como Barretenberg/UltraHonk usam combinações de lookup tables de tamanho moderado e decomposição de inteiros em blocos para implementar tipos $u=8$, $u=32$ e $u=64$ em Noir. Um circuito Noir que usa Field evita esse custo de tabela, mas precisa de mais constraints aritméticas para impor comparações e faixas numéricas; já um circuito que utiliza inteiros nativos delega parte dessa lógica às tabelas de lookup, reduzindo gates aritméticos à custa de mais linhas de tabela e trabalho extra para o provador. O experimento deste TCC, ao comparar Field com $u=8$, $u=32$ e $u=64$, mede empiricamente esse trade-off: a troca entre

complexidade polinomial pura e custo de lookup arguments em UltraHonk, em termos de proving time, tamanho da prova, contagem de constraints e consumo de Gas na EVM.[18][20][62][5][59][9][28]

✓ Fontes desta subseção verificadas.

2.4 A Linguagem Noir e o Ecossistema Aztec

Noir é uma **linguagem de domínio específico (DSL)** para construção de circuitos de conhecimento zero, desenvolvida pela Aztec Labs e disponibilizada como projeto open-source. A linguagem é projetada para abstrair a complexidade criptográfica, permitindo que desenvolvedores escrevam lógica de prova em um estilo de alto nível, enquanto o compilador gera uma representação aritmética adequada para diferentes sistemas de prova. Levantamentos recentes de DSLs para ZK posicionam Noir ao lado de Circom, Cairo, o1js, Leo e Lurk, destacando seu foco em ergonomia, backend-agnosticidade via ACIR e integração profunda com o ecossistema Aztec.[20][64][65][66][8]

Circom oferece uma sintaxe orientada diretamente à descrição de constraints aritméticas, com forte integração a Groth16 e um fluxo de trabalho centrado em arquivos de circuito e templates. Cairo foi projetada para provar execução de programas de propósito geral via STARKs, priorizando escalabilidade e compatibilidade com rollups específicos. Já o1js (SnarkyJS) fornece uma API em TypeScript para construção de circuitos SNARK integrados a aplicações web. Noir diferencia-se por combinar uma sintaxe próxima a linguagens de propósito geral com um pipeline baseado em ACIR, que desacopla a linguagem do backend de prova, e por oferecer integração nativa com ferramentas como Barretenberg e a Aztec Network.[65][67][68][8][20][21]

Arquitetura de compilação: Noir → ACIR → Barretenberg

A documentação oficial de Noir descreve um pipeline em duas etapas: primeiro, o código Noir é compilado em **ACIR** (Abstract Circuit Intermediate Representation), uma representação intermediária que expressa constraints aritméticas de forma backend-agnóstica; em seguida, ACIR é traduzido para circuitos específicos de backends como Barretenberg. ACIR funciona como camada de abstração entre DSL e backend: o mesmo programa Noir pode ser provado por diferentes sistemas, desde que existam conversores de ACIR para os respectivos formatos.[68][8][4]

No ecossistema Aztec, Noir é o front-end oficial e Barretenberg o backend padrão, implementando UltraPlonk, UltraHonk e variantes relacionadas. Ferramentas como nargo geram ACIR e artefatos auxiliares, enquanto a CLI bb consome esses arquivos para produzir provas, chaves de verificação e contratos verificadores Solidity via bb write_solidity_verifier. O TCC utiliza exatamente esse fluxo: o circuito de Range Proof é escrito em Noir, compilado para ACIR sob nargo, e Barretenberg/UltraHonk é empregado para gerar provas e o contrato verificador que será implantado na rede de testes Sepolia.[5][6][8][9][20][28][68]

UltraHonk, Sumcheck e geração de verificadores

Barretenberg é uma biblioteca criptográfica em C++ que implementa uma variedade de primitivas para ZKP, incluindo sistemas de prova PLONKish, compromissos polinomiais, gadgets de alto nível e suporte à recursão. A documentação do projeto e análises especializadas descrevem UltraHonk como um protocolo que combina ideias de PLONK com um protocolo de Sumcheck otimizado, visando melhor escalabilidade em memória para circuitos grandes. Ao substituir o polinômio quociente global por Sumcheck, UltraHonk evita a materialização de polinômios de alto grau em memória, o que reduz o pico de uso de memória e melhora a eficiência em dispositivos com recursos limitados.[69][24][26][6][9][25]

Essa característica é particularmente relevante quando o provador roda em ambientes restritos como navegadores, onde o heap WASM32 e as políticas de alocação são mais rígidos do que em binários nativos. No presente TCC, o Eixo A explora precisamente essa dimensão: ao comparar o proving time de UltraHonk em execução nativa e via WASM32/V8, sob ambiente experimental cuidadosamente controlado, avalia-se se as otimizações de Sumcheck e layout de memória do backend se traduzem em ganhos efetivos sob restrições de heap e JIT.[70][71][29][9][28]

WASM32 como alvo de execução cliente

Para suportar geração de provas no lado cliente, Noir e Barretenberg expõem bindings para **WebAssembly (WASM32)**, permitindo que navegadores executem o provador diretamente. Tutoriais oficiais mostram como usar `@noir-lang/noir_js` e `@aztec/bb.js` para compilar e rodar provas no navegador, consumindo artefatos ACIR e chamando Barretenberg compilado para WASM.[71][8][68]

WebAssembly foi definido pelo W3C como um formato de código de baixo nível, seguro e portátil, projetado como alvo de compilação para linguagens como C/C++ e Rust. Motores como V8 implementam um pipeline com dois estágios: o compilador baseline Liftoff traduz o bytecode WASM rapidamente em código nativo, proporcionando startup rápido, enquanto o compilador otimizador TurboFan recompila funções quentes com otimizações mais agressivas ao longo da execução. Estudos empíricos mostram que, embora WASM ofereça vantagens de desempenho sobre JavaScript em muitas cargas de trabalho, aplicações intensivas em CPU e memória, como provadores ZK, podem sofrer overheads significativos devido a limites de heap, coleta de lixo e custos de sandbox.[29][72][73][74][75][76][77]

O modelo de concorrência do WebAssembly também difere substancialmente do de programas nativos: a especificação define operações atômicas e memória linear compartilhada, mas **não define uma API de criação de threads**, delegando essa responsabilidade ao ambiente hospedeiro. Em navegadores, a paralelização é obtida combinando WebAssembly com **Web Workers** e **SharedArrayBuffer**: o código JavaScript principal instancia workers, compartilha um objeto `WebAssembly.Memory` com a flag `shared: true` e coordena o trabalho via filas de

mensagens e primitivos Atomics. Comentários de implementadores e documentação de ferramentas como Emscripten explicitam que, do ponto de vista de WASM, "threads" são efetivamente a combinação de Web Workers, SharedArrayBuffer e instruções atômicas; o agendamento real é realizado pelo runtime do navegador.[78][79][80][81][72][73][82][83]

Em contraste, o provador nativo de Barretenberg pode explorar diretamente threads do sistema operacional, com controle explícito de afinidade de CPU via taskset e do número de threads via OMP_NUM_THREADS, como previsto na metodologia experimental deste TCC. Ainda que o experimento iguale o número de threads usando navigator.hardwareConcurrency no navegador e o mesmo valor no ambiente nativo, os custos de criação de workers, sincronização via Atomics e troca de contexto na sandbox do navegador são distintos dos custos de threads POSIX ou equivalentes. O Eixo A mede empiricamente o impacto desses dois modelos de concorrência — WebAssembly + Web Workers versus threads nativas — sobre o proving time do UltraHonk.[81][83][84][70][9][28][29]

✓ Fontes desta subseção verificadas.

2.5 Privacidade em Blockchains Públicas: Estado da Arte e Trabalhos Relacionados

Blockchains públicas como o Ethereum foram projetadas com forte ênfase em transparência: cada transação, evento e modificação de estado é replicada por todos os nós, permitindo verificação independente por qualquer participante. Essa transparência é um pilar da segurança e da resistência à censura, mas entra em conflito direto com requisitos de confidencialidade de dados em aplicações financeiras e regulatórias. Provas de solvência, verificações de saldo mínimo ou comprovações de atributos pessoais (como idade ou renda) anteriormente dependiam de divulgação direta de dados; replicar isso em uma blockchain pública frequentemente viola normas de sigilo bancário e leis de proteção de dados.[33][35][57][4]

ZKPs oferecem um caminho para reconciliar transparência de verificação com privacidade de dados, permitindo que propriedades de dados privados sejam atestadas publicamente sem revelar os dados brutos. Contudo, a integração prática de ZKPs impõe novos custos de proving time, complexidade de contratos verificadores e consumo de Gas, especialmente em um cenário pós-Pectra em que EIPs como 170, 4844 e 7623 alteram a economia de armazenamento e de calldata.[35][37][39][57][4]

Abordagens existentes para privacidade on-chain

- **Mixers e coinjoin:** protocolos de mistura, como Tornado Cash, combinam múltiplas entradas e saídas para romper o vínculo observável entre remetente e destinatário. Em muitos casos, essas soluções não escondem valores e acabam sujeitas a restrições regulatórias por seu uso potencial em lavagem de dinheiro.[53][4]

- **Canais de estado off-chain:** canais de pagamento e state channels movem a maior parte das interações para fora da cadeia, mantendo apenas checkpoints agregados on-chain. Essa abordagem reduz o volume de dados expostos, mas nem sempre oferece privacidade forte, pois ainda é possível inferir informações a partir dos estados finais e de metainformações.[29][4]
- **Rollups com privacidade:** rollups ZK com foco em privacidade, como Aztec, combinam execução privada fora da L1 com provas de validade concisas publicadas na cadeia base. Nesses sistemas, transações individuais são agrupadas e provadas como um lote, de forma que a L1 só enxerga um compromisso e uma prova, não os detalhes de cada operação.[85][9][20]
- **Criptomoedas de privacidade dedicadas:** projetos como Monero adotam mecanismos de privacidade 'by design', como RingCT, para esconder remetente, destinatário e quantias em todas as transações, mas sacrificam a expressividade de smart contracts generalistas.[56][54][53]

O TCC se posiciona como uma ponte entre essas linhas: o objetivo é estudar **Range Proofs de atributos privados diretamente em contratos inteligentes EVM**, alinhando-se conceitualmente a soluções de conformidade e solvência, mas mantendo a verificação on-chain na própria Ethereum (ou testnets equivalentes), ao invés de delegar a privacidade integralmente a uma camada 2 dedicada.[7][39][35]

Trabalhos relacionados em benchmarking de ZKPs

- **Benarroch et al. – 'A Benchmarking Framework for (Zero-Knowledge) Proof Systems'** propõem um framework padronizado para avaliação de sistemas de prova, com foco em comparabilidade entre bibliotecas. O trabalho descreve métricas como proving time, verification time, tamanho da prova, uso de memória e largura de banda, e mostra experimentos com diferentes sistemas de prova em ambientes controlados.[57]
- **Ernstberger et al. – 'zk-Bench: Performance Benchmarking of SNARKs'** introduzem uma ferramenta que permite comparar sistemas como Groth16 e PLONK, medindo proving time, custo de verificação e contagem de constraints em diversos problemas. Os autores discutem como parâmetros de circuito e configurações de backend influenciam os resultados, oferecendo um ponto de partida para estudos como o deste TCC.[86]
- **Derei – 'Accelerating the PlonK zkSNARK Proving System using GPU Architectures'** investiga a aceleração do provador PLONK usando GPUs NVIDIA. O trabalho apresenta otimizações de paralelização, mede proving time em diferentes configurações de hardware e discute ganhos de desempenho em comparação à execução apenas em CPU, evidenciando a sensibilidade dos provedores a características de arquitetura de hardware.[87][88]
- **Sheybani et al. – 'Zero-Knowledge Proof Frameworks: A Survey'** apresentam um panorama abrangente de bibliotecas e frameworks ZK,

descrevendo arquiteturas internas e incluindo resultados experimentais de benchmarking. O survey discute métricas, cenários de teste e limitações de diversos sistemas, mas com foco na diversidade de frameworks, não em uma DSL específica ou em um backend como UltraHonk.[4]

Não foram localizados trabalhos com exatamente o perfil deste TCC, isto é, **benchmarking sistemático de provedores Noir/UltraHonk em execução nativa versus WASM32, com controle explícito de bit-width (\$u8\$, \$u32\$, \$u64\$) e instrumentação de Gas em contratos verificadores EVM sob EIPs 170, 4844 e 7623**. Os trabalhos existentes medem sistemas como Groth16, PLONK, Halo2 ou frameworks genéricos, frequentemente em ambiente nativo, com foco em comparações entre sistemas ou aceleração por hardware especializado, e não em DSLs modernas com pipeline ACIR + UltraHonk.[88][86][57][4]

Gap de pesquisa

O **gap de pesquisa** pode ser sintetizado da seguinte forma: apesar da existência de estudos de benchmarking de sistemas ZK, falta um trabalho que examine **detalhadamente o impacto conjunto** de (i) escolha de tipos numéricos (Field vs. \$u8\$, \$u32\$, \$u64\$), (ii) modelo de concorrência e arquitetura de execução (nativo vs. WASM32 + Web Workers + SharedArrayBuffer) e (iii) restrições concretas de EVM (EIP-170, EIP-4844, EIP-7623) sobre proving time, tamanho de prova, contagem de constraints e Gas de verificação de provas UltraHonk geradas a partir de circuitos Noir. O presente TCC preenche essa lacuna ao desenhar um experimento de engenharia de software que isola essas variáveis, aplicando instrumentação de baixo nível e testes estatísticos robustos (Brunner–Munzel com correção de Holm–Bonferroni) para quantificar o custo da privacidade no ecossistema Noir/Barretenberg/EVM.[8][28][37][39][5][7][35][57][4]

✓ Fontes desta subseção verificadas.

2.6 WebAssembly (WASM) em Aplicações Criptográficas

WebAssembly (WASM) é um formato de código de baixo nível, seguro e portátil, padronizado pelo W3C como alvo de compilação para linguagens como C, C++ e Rust. O objetivo é permitir que programas de alto desempenho sejam executados em navegadores e outros ambientes com desempenho próximo ao nativo, dentro de um sandbox que garante isolamento de memória e segurança. A especificação define uma memória linear por módulo e um conjunto de instruções bem tipado, com integração a APIs externas via imports.[72][73][74]

Motores como o V8 adotam um pipeline de compilação em duas camadas: o compilador baseline Liftoff gera rapidamente código nativo a partir do bytecode WASM, otimizando tempo de carregamento, enquanto o compilador otimizador TurboFan recompila funções 'quentes' usando análises multi-passo e otimizações agressivas. Essa combinação busca equilibrar fast startup e desempenho sustentado, mas introduz um período de cold-start em que o código ainda não está completamente otimizado.[75][76][29]

Estudos experimentais sobre desempenho de aplicações WebAssembly mostram que, embora WASM ofereça vantagens sobre JavaScript em diversos cenários, cargas de trabalho intensivas em CPU e memória, como operações criptográficas de alto volume, podem enfrentar padrões de desempenho complexos. Yan et al., em um estudo apresentado no IMC 2021, observaram que o ganho de WASM em relação a JavaScript é altamente dependente de fatores como padrão de acesso à memória, tamanho do heap, políticas de alocação e tiering do compilador, e que aplicações WASM podem consumir significativamente mais memória do que suas versões em JS em alguns casos.[77][29]

Em workloads criptográficos, como geração de provas ZK, o provador precisa executar operações de alta complexidade, incluindo FFTs, multiplicações em campos finitos, operações em curvas elípticas e manipulação de grandes vetores de dados. Essas operações exercem forte pressão sobre o heap e a largura de banda de memória, tornando o impacto de limites de memória de WASM32 (tipicamente em torno de alguns gigabytes, salvo extensões como Memory64) e do tratamento de garbage collection particularmente relevantes. Documentação e artigos recentes destacam que o uso de **SharedArrayBuffer** e multi-threading em WASM (via Web Workers) pode oferecer ganhos de até 8x em workloads fortemente paralelizáveis, mas exige cross-origin isolation e configuração cuidadosa de cabeçalhos de segurança.[73][83][70][81][86][88][29][57]

Para suportar multithreading, o ecossistema web combina três componentes: SharedArrayBuffer, instruções atômicas de WASM e Web Workers, com a orquestração feita por bibliotecas como Emscripten. Na prática, isso significa que o custo de criar e coordenar threads WASM inclui overhead de criação de workers, passagem de mensagens e sincronização via Atomics, diferindo significativamente do custo de threads nativas em C++.[79][80][78][81]

No contexto deste TCC, o provador UltraHonk é executado em dois ambientes: (i) uma CLI nativa de Barretenberg, configurada com OMP_NUM_THREADS e afinidade de CPU via taskset, e (ii) um provador compilado para WASM32 rodando em navegador, com número de threads inferido de navigator.hardwareConcurrency e paralelismo realizado via Web Workers e SharedArrayBuffer, quando suportado. Ao fixar parâmetros como número de threads e políticas de frequência de CPU, o experimento isola o impacto específico do runtime WASM e de seu modelo de concorrência sobre o proving time do UltraHonk, mensurando a dominância estocástica entre execuções nativas e WASM32 por meio do teste de Brunner–Munzel. Essa comparação é crucial para avaliar a viabilidade de aplicações ZK client-side em dApps Ethereum, nas quais o navegador é frequentemente o ambiente predominante de geração de provas.[70][71][81][28][68][29][4]

✓ Fontes desta subseção verificadas.

7. METODOLOGIA DE PESQUISA

A presente pesquisa propõe o desenvolvimento, a implementação e a avaliação empírica de desempenho de um circuito criptográfico do tipo Range Proof construído no ecossistema Noir. O objetivo central é avaliar empiricamente o custo computacional e a degradação de desempenho deste circuito, quantificando o impacto arquitetural do ambiente de execução (Nativo vs. WASM32) e o peso criptográfico da imposição de diferentes larguras de bits (\$u8\$, \$u32\$, \$u64\$) sobre o tempo de geração da prova, tamanho do artefato e custo de verificação na EVM. O delineamento metodológico está estruturado para garantir a validade científica da avaliação perante as limitações práticas de infraestrutura Web3 (EVM pós-Pectra e arquitetura WASM32).

3.1 Classificação da Pesquisa e Fixação Tecnológica (Stack Pinning)

Para garantir a reprodutibilidade e evitar a quebra de compatibilidade em um ecossistema em rápida evolução, o ambiente de desenvolvimento é estritamente fixado:

- **Compilador de Circuito:** Noir CLI (nargo versão $\geq 0.38.0$).
- **Backend Criptográfico e Gerador de Verificador:** Barretenberg CLI (bb versão correspondente ao nargo), operando sob o esquema **UltraHonk**.
- **Alvo de Deploy e Instrumentação:** Rede de testes Sepolia (Ethereum). Para fins de instrumentação rigorosa de Gas, as transações de verificação serão simuladas em um fork local espelhando o estado exato do bloco mais recente da Sepolia (via anvil), garantindo equivalência com a rede pública, porém isolando a coleta métrica contra restrições de rate limit e ruídos de mempool.

A pesquisa caracteriza-se como **Aplicada com Avaliação de Desempenho**, orientada por métricas quantitativas e testes de dominância estocástica robustos a ambientes ruidosos.

3.2 Formulação das Hipóteses Estatísticas e Tamanho do Efeito

A testagem estatística será restrita às variáveis empíricas que apresentam flutuação estocástica (Tempo de Geração da Prova). Variáveis de natureza determinística não serão submetidas a testes de hipóteses, sendo analisadas via comparação analítica exata.

Devido à natureza multimodal da latência no ambiente WASM32, a avaliação das hipóteses empíricas será amparada por testes não paramétricos de igualdade estocástica (Teste de Brunner-Munzel). **Neste delineamento, a própria probabilidade estocástica reportada pelo teste ($P(X < Y)$) será adotada formalmente como a métrica de Tamanho do Efeito ([i]Effect Size[/i]), medindo a significância prática da degradação além da mera significância estatística.**

Eixo A: Impacto Arquitetural do Ambiente de Execução

Avalia a variável estocástica (Tempo de Prova) frente ao ambiente de execução.

- **Hipótese Nula (\$H_0\$):** Não há diferença estocástica no tempo de geração da prova entre a execução nativa e a execução via WebAssembly. A probabilidade de uma prova gerada nativamente (\$X\$) ser mais rápida do que uma gerada em WASM (\$Y\$) é igual a 0,5 ($P(X < Y) + 0.5 \cdot P(X = Y) = 0.5$).
- **Hipótese Alternativa (\$H_1\$):** Existe dominância estocástica da execução nativa sobre o ambiente WASM. O ambiente web apresenta latência estatisticamente superior ($P(X < Y) + 0.5 \cdot P(X = Y) \neq 0.5$).

Eixo B: Custo Empírico da Restrição de Intervalo (Bit-width)

- **Hipótese Nula (\$H_0\$):** A imposição de limites de intervalo estritos (\$u8\$, \$u32\$, \$u64\$) não gera diferença estatisticamente significativa no tempo de geração da prova em comparação a um circuito de linha de base (Field).
- **Hipótese Alternativa (\$H_1\$):** A restrição da largura de bits incrementa significativamente a latência de geração da prova em relação à linha de base, refletindo o overhead computacional das Lookup Tables do UltraHonk.

3.3 Delineamento Experimental e Variáveis de Controle

O escopo isola lógicas de negócios complexas focando na instrumentação das seguintes variáveis independentes e controles sistêmicos:

- **Controle de Largura de Bits e Linha de Base:** Variando as restrições de inteiros (\$u8\$, \$u32\$, \$u64\$) contra um circuito idêntico utilizando o tipo matemático irrestrito Field para isolar o custo da restrição de intervalo.
- **Controle Simétrico de Concorrência (Threads) e Topologia Física:** Para garantir o isolamento arquitetural e evitar vieses de multiplexação, o paralelismo será orquestrado de forma simetricamente controlada. O front-end (Chrome, versão exata registrada nos logs de teste) definirá o limite de threads baseando-se em `navigator.hardwareConcurrency` (valor \$N\$). O provador WASM será instanciado com este limite exato (`Barretenberg.new({ threads: N })`). Em contrapartida, a CLI nativa será ancorada ao mesmo valor via `export OMP_NUM_THREADS=N` acoplada ao comando `taskset`. Antes da execução, a topologia de hardware será mapeada (via `lscpu --extended`) e a máscara de afinidade do taskset será configurada para isolar \$N\$ núcleos físicos distintos, evitando a canibalização de recursos por threads SMT/Hyper-Threading na mesma ALU.
- **Controle Térmico (CPU Throttling):** Para expurgar a variância imposta por estrangulamento térmico durante os testes contínuos nativos, o governador da CPU será fixado no nível do kernel Linux (`cpupower frequency-set --governor performance`) e as funções de autoboot de hardware (Intel Turbo Boost/AMD Core Performance Boost) serão temporariamente suprimidas via `sysfs`, garantindo uma frequência base de relógio inalterada ao longo da coleta.

Fase 1: Construção e Deploy sob EIP-170

Compilação do código Noir (ACIR) seguida pela geração do Smart Contract via `bb write_solidity_verifier`. Para transpor o limite de tamanho da EVM (EIP-170, max 24,5 KB), o artefato será submetido a tuning agressivo (`optimizer_runs = 1`). Caso a complexidade invalide o otimizador, adotar-se-á a topologia Split-Verifier.

Fase 2: Instrumentação Estatística e Isolamento de Ambiente

8. **Protocolo de Aquecimento (Burn-in e Mitigação de Cold-Start):** Execução inicial de $K = 15$ provas descartadas. No WASM32, isso força o tier-up do motor V8 (transição do Liftoff para o TurboFan), isolando a latência criptográfica do overhead de compilação JIT.
9. **Amostragem e Randomização em Blocos:** Registro de $N = 100$ ciclos independentes por condição experimental. Não haverá limpeza forçada de memória para preservar a validade ecológica do uso contínuo de heap. A ordem de execução (Field, $u8$, $u32$, $u64$) será rigorosamente randomizada em blocos.
10. **Parametrização, Erro e Falhas Sistêmicas (Timeout Dinâmico):** Significância basal de $\alpha = 0,05$ com correção de Holm-Bonferroni para múltiplas comparações (Eixo B). Para isolar lentidão criptográfica de falhas de ambiente (ex: GC thrashing excessivo ou Out of Memory no V8), um **timeout dinâmico** será estabelecido para cada condição como $t_{\max} = 10 \times \tilde{x}_{\text{burn-in}}$ (dez vezes a mediana da latência obtida no respectivo burn-in). Condições experimentais que excederem a tolerância de falhas em mais de 15% da amostra ($n < 85$) serão descontinuadas e reportadas sob a métrica de Failure Rate.

Fase 3: Auditoria de Correção Lógica (Qualitativa)

Execução de testes negativos (Under-constrained Testing) submetendo inputs fora do intervalo, assegurando ausência de vulnerabilidades lógicas antes da coleta de desempenho.

3.4 Variáveis e Métricas de Avaliação Quantitativa

A viabilidade do sistema será determinada por dois grupos de métricas.

Métricas Empíricas (Avaliação Estocástica):

11. **Tempo de Geração da Prova ([i]Proving Time[/i]):** Mensurado em milissegundos via cliente (Prover), analisado por Gráficos de Violino, Funções de Distribuição Cumulativa Empírica (ECDF) e Teste de Brunner-Munzel.

Métricas Determinísticas (Avaliação Comparativa Exata):

12. **Custo Computacional e de Dados ([i]Gas Cost EVM Pectra[/i]):** Avaliado em ambiente estéril no Fork Local da rede Sepolia (anvil), operando de maneira autônoma sobre o bloco mais recente.

- Execution Gas: Custo puro das operações de validação matemática.
Aviso (Disclaimer): Devido à necessidade arquitetural de burlar a restrição do EIP-170 utilizando `optimizer_runs = 1`, esta métrica representará um cenário contratual desotimizado operacionalmente. O Gas extraído não reflete o limite teórico matemático de eficiência do esquema UltraHonk, mas sim um teto pessimista exigido pela limitação da camada de execução atual.
 - Calldata Gas (EIP-7623): Custo operacional padrão via calldata, majorado pelas regras da EVM Pectra.
 - Blob Gas (EIP-4844): Tratada restritamente como uma **projeção teórica de transporte**. Como Smart Contracts não possuem acesso ao escopo de dados de blobs na camada de execução (apenas ao hash), a verificação on-chain isolada da Range Proof será impossibilitada via EIP-4844. O cálculo fracionário servirá apenas para avaliar viabilidade econômica em infraestruturas onde o circuito opere de forma acoplada em Camada 2 (Rollups/Sequencers).
13. **Tamanho da Prova ([i]Proof Size[/i]):** Tamanho do payload submetido (Bytes).
14. **Complexidade do Circuito ([i]Constraint Count[/i]):** Contagem determinística de ACIR Opcodes (nargo info) e Backend Gates (bb gates) para extrair a topologia real fornecida ao Barretenberg.

8. CRONOGRAMA

A seguir apresenta-se o cronograma de execução do TCC, abrangendo o período de junho a novembro de 2026 (seis meses). As etapas seguem a sequência lógica do delineamento metodológico descrito na Seção 7, com sobreposições previstas entre fases de implementação e redação.

Etapa	Jun/2026	Jul/2026	Ago/2026	Set/2026	Out/2026	Nov/2026
Revisão e refinamento bibliográfico	•	•				
Implementação do circuito em Noir (Field, u8, u32, u64)		•	•			
Deploy e auditoria lógica (under-constrained testing)			•			
Coleta de dados: benchmarking nativo			•	•		
Coleta de dados: benchmarking WASM32				•		
Instrumentação de Gas no fork Sepolia (anvil)				•	•	
Análise estatística (Brunner-Munzel, Holm-Bonferroni, ECDF, violin plots)					•	•
Redação do TCC		•	•	•	•	•
Revisão final e submissão						•

A execução paralela das atividades de redação ao longo de todo o período reflete a natureza iterativa da produção científica, em que registros parciais são elaborados concomitantemente à coleta e análise de dados. A revisão final e a submissão concentram-se no mês de novembro de 2026.

9. REFERÊNCIAS

(ABNT NBR 6023:2018)

- AZTEC LABS. Aztec's Core Cryptography, Now in Noir. Aztec Network Blog, 2026. Disponível em: <https://aztec.network/blog/aztecs-core-cryptography-now-in-noir>. Acesso em: maio 2026. [20]
- BARRETEMBERG / AZTEC. Getting Started | Barretenberg. 2026. Disponível em: https://barretenberg.aztec.network/docs/getting_started/. Acesso em: maio 2026. [6]
- BARRETEMBERG / L2BEAT. Barretenberg – L2BEAT Entry. 2026. Disponível em: <https://l2beat.com/zk-catalog/barretenberg>. Acesso em: maio 2026. [9]
- BENARROCH, Daniel et al. A Benchmarking Framework for (Zero-Knowledge) Proof Systems. [S.l.: s.n.], 2020. Disponível em: <https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-benchmarking.pdf>. Acesso em: maio 2026. [57]
- BÜNZ, Benedikt et al. Bulletproofs: Short Proofs for Confidential Transactions and More. In: IEEE SYMPOSIUM ON SECURITY AND PRIVACY, 2018. Anais [...]. [S.l.: s.n.], 2018. Disponível em: <https://ieeexplore.ieee.org/document/8418611/>. Acesso em: maio 2026. [48][49]
- BUTERIN, Vitalik. Ethereum Whitepaper: A Next Generation Smart Contract & Decentralized Application Platform. [S.l.: s.n.], 2013. Disponível em: <https://ethereum.org/whitepaper/>. Acesso em: maio 2026. [34][33]
- CANTINA. EIP-7623 – Increase Calldata Cost. Cantina Docs, 2024. Disponível em: <https://cantina.xyz/introduction/pectra-competition-resources/eip-7623>. Acesso em: maio 2026. [47][44]
- DEREI, Tal. Accelerating the PlonK zkSNARK Proving System using GPU Architectures. 2023. Master's Thesis (Mestrado em Ciência da Computação) – Lehigh University, 2023. Disponível em: https://bpb-us-w2.wpmucdn.com/wordpress.lehigh.edu/dist/0/2548/files/2023/05/Master_Thesis_Tal_Derei.pdf. Acesso em: maio 2026. [87][88]
- ETHEREUM IMPROVEMENT PROPOSALS. EIP-170: Contract code size limit. 2016. Disponível em: <https://eips.ethereum.org/EIPS/eip-170>. Acesso em: maio 2026. [35]
- ETHEREUM IMPROVEMENT PROPOSALS. EIP-4844: Shard Blob Transactions. 2022. Disponível em: <https://eips.ethereum.org/EIPS/eip-4844>. Acesso em: maio 2026. [39]

- ETHEREUM IMPROVEMENT PROPOSALS. EIP-7623: Increase calldata cost. 2024. Disponível em: <https://eips.ethereum.org/EIPS/eip-7623>. Acesso em: maio 2026. [37]
- FIAT, Amos; SHAMIR, Adi. Fiat–Shamir heuristic. In: CRYPTO '86. [S.l.: s.n.], 1986. Disponível em: https://en.wikipedia.org/wiki/Fiat-Shamir_heuristic. Acesso em: maio 2026. [10][11]
- GABIZON, Ariel; WILLIAMSON, Zac; CHIESA, Alessandro. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge. [S.l.: s.n.], 2019. Disponível em: <https://research.protocol.ai/publications/plonk-permutations-over-lagrange-bases-for-oecumenical-noninteractive-arguments-of-knowledge/>. Acesso em: maio 2026. [17][15][18]
- GOLDWASSER, Shafi; MICALI, Silvio; RACKOFF, Charles. The Knowledge Complexity of Interactive Proof Systems. SIAM Journal on Computing, [s.l.], 2006. Disponível em: <https://epubs.siam.org/doi/10.1137/0218012>. Acesso em: maio 2026. [2][1]
- HALO2 BOOK. Lookup Argument – The Halo 2 Book. Zcash, 2022. Disponível em: <https://zcash.github.io/halo2/design/proving-system/lookup.html>. Acesso em: maio 2026. [89][59]
- ICME. The Lookup Singularity – How Zero-Knowledge Proofs Can Be Supercharged with Lookups. ICME Blog, 2024. Disponível em: <https://blog.icme.io/the-lookup-singularity/>. Acesso em: maio 2026. [62]
- INGONYAMA. Sumcheck 201 – Algebraic Techniques for High-Performance Proof Systems. Ingonyama Blog, 2024. Disponível em: <https://www.ingonyama.com/post/sumcheck-201>. Acesso em: maio 2026. [27][25]
- LAMBDA CLASS. If You Don't Know, Look It Up – How to Create Lookup Tables for Zero-Knowledge Proofs. LambdaClass Blog, 2023. Disponível em: <https://blog.lambdaclass.com/lookups/>. Acesso em: maio 2026. [63]
- LAMBDA CLASS. Optimizing the Sumcheck Protocol: Algebraic Techniques for High Performance. LambdaClass Blog, 2026. Disponível em: <https://blog.lambdaclass.com/optimizing-sumcheck/>. Acesso em: maio 2026. [26]
- MONERO PROJECT. Ring Confidential Transactions. Ledger, 2016. Disponível em: <https://www.ledgerjournal.org/ojs/ledger/article/view/34>. Acesso em: maio 2026. [54][53]
- NOIR LANG. Noir Documentation. 2026. Disponível em: <https://noir-lang.org/docs/>. Acesso em: maio 2026. [68][8]
- PEDERSEN, Torben Pryds. Pedersen Commitments. Síntese em: TOMESCU, Alin. Pedersen Commitments – Blog. 2025. Disponível em: <https://alinush.github.io/pedersen>. Acesso em: maio 2026. [52][50][51]

- QUICKNODE. An Intro to EIP-4844 (Proto-Danksharding). 2025. Disponível em: <https://www.quicknode.com/guides/ethereum-development/transactions/eip4844-explained>. Acesso em: maio 2026. [43]
- RISC ZERO. About STARKs – RISC Zero Developer Docs. 2024. Disponível em: <https://dev.risczero.com/reference-docs/about-starks>. Acesso em: maio 2026. [21]
- SHEYBANI, Nojan et al. Zero-Knowledge Proof Frameworks: A Survey. arXiv preprint, 2025. Disponível em: <https://arxiv.org/html/2502.07063v1>. Acesso em: maio 2026. [4]
- SZABO, Nick. Smart Contracts. 1994. Disponível em: <https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html>. Acesso em: maio 2026. [31][32]
- ULTRAHONK / ZKVERIFY. Ultrahonk Verifier Documentation. zkVerify, 2024. Disponível em: https://docs.zkverify.io/architecture/verification_pallets/ultrahonk. Acesso em: maio 2026. [5]
- V8 TEAM. WebAssembly Compilation Pipeline – V8 JavaScript Engine. 2020. Disponível em: <https://v8.dev/docs/wasm-compilation-pipeline>. Acesso em: maio 2026. [76][75]
- W3C. WebAssembly Core Specification. W3C, 2019–2025. Disponível em: <https://webassembly.org/specs/>. Acesso em: maio 2026. [74][72][73]
- WANG, Weihang et al. Understanding the Performance of WebAssembly Applications. In: INTERNET MEASUREMENT CONFERENCE (IMC), 2021. Anais [...]. [S.l.: s.n.], 2021. Disponível em: <https://dl.acm.org/doi/10.1145/3487552.3487827>. Acesso em: maio 2026. [77][29]
- WEB.DEV / GOOGLE. Using WebAssembly Threads in C/C++ and Rust. web.dev, 2021. Disponível em: <https://web.dev/articles/webassembly-threads>. Acesso em: maio 2026. [83][84]
- WEBASSEMBLY COMMUNITY GROUP. Shared-Everything Threads Proposal. GitHub, 2023. Disponível em: <https://github.com/WebAssembly/shared-everything-threads>. Acesso em: maio 2026. [82]
- ZK LANGUAGE SURVEY. ZK Language Survey: Noir, o1js, Circom, Leo, Cairo, Lurk. Gate.io Post, 2023. Disponível em: <https://www.gate.io/pt-br/post/status/4188889>. Acesso em: maio 2026. [66][65]